

# Granja concurrente de impresión 3D

## Trabajo Práctico 1 — Programación Concurrente y Paralela 2026

Rafael Barrio

Alexis Tomás Garay

Pedro Guzman Gonzalez

Santino Chesta

Valentino Zuchella Paz

Grupo Warrior-of-Interruptions — 2026

## Integrantes y repositorio

| Nombre                 | Documento | Usuario Git |
|------------------------|-----------|-------------|
| Rafael Barrio          | [[TODO]]  | [[TODO]]    |
| Alexis Tomás Garay     | [[TODO]]  | [[TODO]]    |
| Pedro Guzman Gonzalez  | [[TODO]]  | [[TODO]]    |
| Santino Chesta         | [[TODO]]  | [[TODO]]    |
| Valentino Zuchella Paz | [[TODO]]  | [[TODO]]    |

- **Grupo:** Warrior-of-Interruptions
- **Repositorio:** [[TODO: URL]]
- **Tag de entrega:** Warrior-of-Interruptions-entrega-tp1

## 1. Introducción y objetivo

El trabajo simula una granja industrial de impresión 3D. El sistema recibe órdenes de fabricación, les asigna una impresora, valida el modelo, ejecuta la impresión y controla la calidad de la pieza obtenida. Cada orden comienza en `CREATED` y finaliza exactamente en uno de cuatro estados terminales: `APPROVED`, `REJECTED`, `PRINT_FAILED` o `DEFECTIVE`.

Los objetivos son: (i) que las cuatro etapas funcionen **concurrentemente**, (ii) que compartan recursos de manera segura, (iii) que todas las órdenes se procesen conservando la consistencia de sus estados y (iv) que la simulación finalice sin dejar hilos activos ni impresoras reservadas.

El sistema se implementa en Java 21 con Maven. La decisión de validación, impresión y calidad se delega exclusivamente en `OutcomeDecider`, provisto por la cátedra, de modo que el resultado final de cada orden es determinista para una semilla e identificador dados.

## 2. Arquitectura general

La solución organiza el trabajo como un **pipeline de cuatro etapas** comunicadas por colas FIFO. El hilo principal (`main`) crea las órdenes y las deposita en la primera cola; a partir de ahí, los workers avanzan cada orden por el recorrido:



## 2.2 Diagrama de secuencia

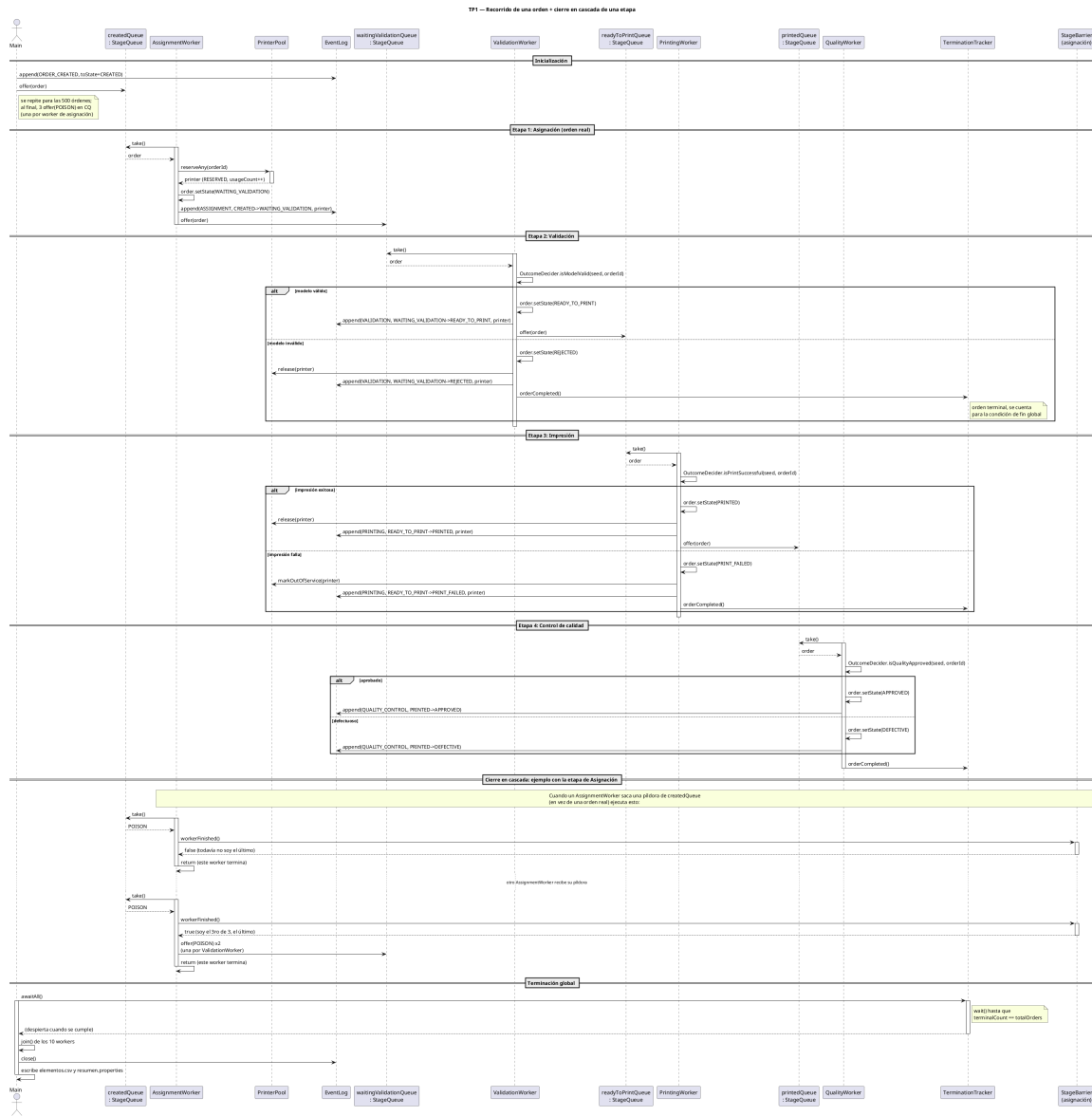


Figura 2: Diagrama de secuencia – inicio conjunto y recorrido de una orden

[[TODO: explicar el arranque conjunto (StartGate) y el recorrido feliz de una orden, incluyendo las salidas anticipadas por rechazo y por fallo de impresión.]]

### 3. Recursos compartidos

Los recursos compartidos y el mecanismo que los protege son:

| Recurso                                 | Clase              | Mecanismo                     | Quién escribe                                                    | Quién lee             | Invariantes              |
|-----------------------------------------|--------------------|-------------------------------|------------------------------------------------------------------|-----------------------|--------------------------|
| Órdenes CREATED es-perando asignación   | createdQueue       | Semaphore + monitor interno   | main (inicial)                                                   | workers de asignación | INV-01, INV-03           |
| Órdenes WAITING_VALIDATION              | ValidationQueue    | Semaphore + monitor interno   | workers de asignación                                            | workers de validación | INV-01, INV-03           |
| Órdenes READY_TO_PRINT                  | PrintingQueue      | Semaphore + monitor interno   | workers de validación                                            | workers de impresión  | INV-01, INV-03           |
| Órdenes PRINTED                         | qualityQueue       | Semaphore + monitor interno   | workers de impresión                                             | workers de calidad    | INV-01, INV-03           |
| Matriz de impresoras y disponibilidad   | PrinterPool        | synchronized + Semaphore      | asignación (reserva); validación/impresión (libera o inhabilita) | los diez workers      | INV-04... INV-08, INV-10 |
| Registro de eventos y contador sequence | EventLog           | synchronized                  | los diez workers + main                                          | —                     | EVT-03, EVT-08           |
| Contador de órdenes terminales          | TerminationTracker | synchronized + wait/notifyAll | validación/impresión/calidad                                     | main                  | TRM-01                   |
| Cierre de cada etapa                    | StageBarrier       | synchronized                  | workers de la etapa                                              | —                     | INV-09, INV-10           |
| Compuerta de arranque                   | StartGate          | synchronized + wait/notifyAll | main y workers                                                   | workers               | GEN-04                   |

Order y Printer **no tienen lock propio**: en todo momento una orden es propiedad de un único worker (el que la retiró de una cola) y una impresora solo se modifica dentro de la sección crítica de PrinterPool. La exclusión mutua la aportan los recursos de la tabla, no los objetos de datos.

## 4. Condiciones de carrera posibles y cómo se evitan

1. **Dos workers de asignación reservan la misma impresora.** Se adquiere primero un permiso del semáforo de impresoras disponibles y luego `reserveFor` busca y marca `RESERVED` dentro de una única sección `synchronized` (INV-04).
2. **Una orden es tomada por dos workers de la misma etapa.** `StageQueue.take()` hace `acquire()` del semáforo y la extracción de la deque como una operación atómica respecto de otros `take/offer` (INV-01, INV-03).
3. **El evento se registra en un orden distinto al de la transición.** Cada worker cambia el estado, registra el evento y recién entonces encola la orden en la etapa siguiente; como la orden tiene un único dueño, no puede intercalarse con otro hilo (EVT-08).
4. **Una orden “se pierde” entre cambiar de estado y encolarse.** El cambio de estado, el registro y el `offer` ocurren en secuencia dentro del mismo worker, sin que la orden pase por otro hilo en el medio.
5. **Una píldora se inyecta antes de encolar todas las órdenes reales.** Las píldoras se encolan detrás de las órdenes en la misma FIFO, y el `StageBarrier` solo confirma el cierre cuando el último worker consumió su píldora, momento en el que ya encoló todo su trabajo.
6. **Se libera una impresora inexistente o no reservada.** `release` y `markOutOfService` verifican el estado previo y lanzan excepción ante una inconsistencia.

## 5. Estrategia de sincronización

### 5.1 Colas entre etapas (StageQueue)

Cada frontera del pipeline tiene una cola FIFO (`ArrayDeque`) protegida por un monitor propio y un `Semaphore` que cuenta los elementos disponibles. El productor inserta el elemento bajo el monitor y luego publica un permiso; el consumidor adquiere un permiso y solo entonces retira el elemento. El semáforo evita la espera activa y modela exactamente “cuántos elementos hay esperando” sin que cada worker reevalúe una condición a mano, evitando además el efecto *thundering herd* de un `notifyAll` sobre una condición.

### 5.2 Pool de impresoras (PrinterPool)

`PrinterPool` administra tanto la matriz como un `Semaphore` de impresoras disponibles. Reservar = adquirir un permiso + marcar `AVAILABLE` → `RESERVED` bajo `synchronized`. Liberar = marcar `RESERVED` → `AVAILABLE` y devolver el permiso. Una impresora que falla pasa a `OUT_OF_SERVICE` y **no** devuelve el permiso, por lo que nunca se reutiliza (INV-07). El semáforo bloquea sin espera activa cuando todas las impresoras están ocupadas, incluso con una sola impresora.

### 5.3 Registro de eventos (EventLog)

El contador `sequence` y la escritura de la fila se realizan bajo un mismo `synchronized`: la asignación del número y el volcado al archivo son una única operación lógica, de modo que la secuencia es creciente, única y respeta el orden de escritura.

### 5.4 Coordinación de arranque y cierre

- `StartGate` (arranque conjunto) y `StageBarrier` (último worker de la etapa) usan monitores.

- TerminationTracker usa wait/notifyAll: el principal espera a que todas las órdenes alcancen un estado terminal o a que algún worker falle.

## 5.5 Por qué no hay locks en Order/Printer

El diseño respeta *ownership* de un solo hilo por objeto en cada instante: una orden vive en exactamente una cola o en la mano de un único worker, y una impresora solo se toca dentro de PrinterPool. Añadir synchronized a los accesos de Order sería redundante y ocultaría el verdadero punto de exclusión mutua.

## 5.6 Justificación de la elección

Se optó por una cola por transición con semáforo + monitor (en lugar de un único lock global o de wait/notify directo sobre las listas) porque cada etapa espera solo por su propia bandeja de entrada sin bloquear a las demás, el modelo es el mismo para las cuatro fronteras (una sola clase reutilizable) y el semáforo encapsula el conteo de elementos disponibles de forma correcta.

## 6. Condiciones de espera y despertar de los hilos

| Hilo                  | Dónde espera                            | Condición de espera                               | Quién lo despierta                                   |
|-----------------------|-----------------------------------------|---------------------------------------------------|------------------------------------------------------|
| Los diez workers      | StartGate.awaitOpen()                   | La compuerta no está abierta                      | main con notifyAll() al confirmar que todos llegaron |
| Los diez workers      | StageQueue.take() → Semaphore.acquire() | No hay elemento en su cola                        | Un productor con release() tras encolar              |
| Workers de asignación | PrinterPool.reserveForNo → acquire()    | No hay impresoras disponibles                     | release() al liberar una impresora                   |
| main                  | TerminationTracker.awaitAll()           | Aún no terminaron todas las órdenes ni hubo fallo | orderCompleted()/fail() con notifyAll()              |
| main                  | WorkerGroup.joinAll()                   | Algún worker sigue vivo                           | La finalización del worker                           |

## 7. Mecanismo de terminación

**Píldoras + barrera, en cascada.** Junto con las órdenes reales se encola una píldora (valor centinela) por cada worker consumidor de la cola. Como las píldoras van detrás de las órdenes reales en la misma FIFO, ningún worker las recibe antes de que se haya encolado todo el trabajo. Al consumir su píldora, cada worker registra su cierre en el StageBarrier de su etapa; el **último** en llegar es el único que fabrica e inyecta las píldoras de la etapa siguiente. La calidad, última etapa, no inyecta nada.

En paralelo, cada vez que una orden llega a un estado terminal (APPROVED, REJECTED, PRINT\_FAILED, DEFECTIVE) el worker correspondiente avisa a TerminationTracker. El principal está bloqueado en awaitAll() y se despierta cuando el contador llega a totalOrders; entonces hace join() de los diez hilos, cierra el registro de eventos y escribe los archivos de resultado. Esto garantiza que al finalizar no queden órdenes en estados intermedios (INV-09) ni impresoras RESERVED (INV-10).

**Cancelación ante fallos.** Si un worker lanza una excepción o se lo interrumpe, el WorkerGroup registra el primer fallo en el TerminationTracker e interrumpe a todos los workers; los que estaban bloqueados en un semáforo o una demora despiertan con InterruptedException y terminan. El principal espera su cierre y relanza el fallo. No se usa Thread.stop ni System.exit.

## 8. Determinismo

La decisión de cada etapa se toma exclusivamente con OutcomeDecider, cuyos resultados dependen solo de la semilla de configuración y del identificador de la orden. Como cada orden recorre el pipeline exactamente una vez (asignación, validación, impresión y calidad a lo sumo una vez), los estados finales son independientes del orden en que se entrelacen los hilos. No se exige igualdad en la impresora asignada, los contadores individuales ni los tiempos.

## 9. Análisis teórico del tiempo de ejecución

El sistema es un pipeline (línea de ensamble). Sea  $d_i$  la demora por orden de la etapa  $i$  y  $W_i$  su cantidad de workers. En régimen estacionario, la etapa  $i$  procesa una orden cada  $d_i / W_i$ ; el **ciclo del sistema** lo fija la etapa más lenta:

$$\text{ciclo} = \max_i ( d_i / W_i )$$

El tiempo total aproximado es la latencia de llenado más el ciclo por cada orden restante:

$$\text{makespan} \approx ( \sum_i d_i ) + (N - 1) \cdot \text{ciclo}$$

Con la configuración oficial ( $N = 500$ ,  $d = 50/120/180/80$  ms,  $W = 3/2/3/2$ ):

| Etapa      | $d_i$ (ms) | $W_i$ | $d_i / W_i$ (ms) |
|------------|------------|-------|------------------|
| Asignación | 50         | 3     | 16.7             |
| Validación | 120        | 2     | <b>60</b>        |
| Impresión  | 180        | 3     | <b>60</b>        |
| Calidad    | 80         | 2     | 40               |

Validación e impresión son **cuellos de botella conjuntos** (60 ms por orden). Predicción:

$$\text{makespan} \approx 430 + 499 \cdot 60 = 30\,370 \text{ ms} \approx 30.4 \text{ s}$$

**Predicciones del barrido.** Como validación e impresión están empatadas como cuello de botella, agregar un worker a *una sola* de ellas no debería mejorar el tiempo (la otra sigue en 60 ms); agregarlo a *ambas* sí lo reduce. Algunas predicciones:

| Configuración (asig/val/imp/cal) | ciclo teórico | makespan teórico      |
|----------------------------------|---------------|-----------------------|
| 1/1/1/1                          | 180 ms        | ≈ 90.3 s              |
| 2/2/2/2                          | 90 ms         | ≈ 45.3 s              |
| 3/2/3/2 (oficial)                | 60 ms         | ≈ 30.4 s              |
| 3/4/3/2                          | 60 ms         | ≈ 30.4 s (sin mejora) |
| 3/2/4/2                          | 60 ms         | ≈ 30.4 s (sin mejora) |
| 3/4/4/2                          | 45 ms         | ≈ 22.9 s              |
| 4/4/4/4                          | 45 ms         | ≈ 22.9 s              |

## 10. Resultados experimentales

### 10.1 Protocolo

- Configuración oficial salvo la variable que se modifique en cada barrido.
- Semilla fija, de modo que los estados finales son idénticos entre corridas y solo varía el tiempo.
- Cinco corridas por variante; se descarta la primera (calentamiento de la JVM) y se reporta mediana (mín.-máx.) de `durationMillis`.
- [[TODO: describir máquina, SO, versión de Java.]]

### 10.2 Corrida oficial

[[TODO: tabla con las 5 corridas: `durationMillis` y distribución de estados finales (approved/rejected/printFailed/defective).]]

| Corrida   | <code>durationMi-</code><br><code>llis</code> | <code>approved</code> | <code>rejected</code> | <code>printFailed</code> | <code>defective</code> |
|-----------|-----------------------------------------------|-----------------------|-----------------------|--------------------------|------------------------|
| 1 (desc.) | 31 792                                        | 359                   | 71                    | 45                       | 25                     |
| 2         | [[TODO]]                                      |                       |                       |                          |                        |
| ...       |                                               |                       |                       |                          |                        |

### 10.3 Comparación teórico vs observado

[[TODO: comparar 30.4 s teóricos contra la mediana observada (~31.8 s) y explicar el sobre costo (arranque de la JVM, contención del log de eventos, colas, scheduling).]]

### 10.4 Efecto de cambiar la cantidad de hilos

[[TODO: pegar tabla del barrido de hilos y contrastar con las predicciones de la sección 9, en particular que subir un solo cuello de botella no mejora.]]

### 10.5 Efecto de cambiar las demoras

[[TODO: pegar tabla del barrido de demoras (×0, ×0.5, ×1, ×2) y verificar la proporcionalidad; señalar qué etapa domina.]]

## 11. Justificación de las decisiones de diseño

[[TODO: resumir los trade-offs del ADR: opción elegida (cola por transición) frente a lock global y frente a wait/notify; ownership de un solo hilo por objeto; píldoras + barrera en lugar de un conteo fijo de órdenes por etapa; uso de OutcomeDecider como única fuente de decisión.]]

## 12. Limitaciones y trabajo futuro

- El registro de eventos es un punto de serialización global; con demoras reales no resultó cuello de botella, pero es una línea para medir con más carga.
- No se resuelve el agotamiento de impresoras porque las configuraciones de evaluación garantizan que no ocurre.

## 13. Conclusiones

[[TODO: síntesis de correctitud (invariantes), desempeño (coincidencia teoría/medición) y aprendizajes sobre sincronización.]]

## Anexo A. Compilación y ejecución

```
mvn clean test          # suite pública + tests del grupo
mvn clean package       # genera target/tp1.jar
java -jar target/tp1.jar
```

La aplicación siempre lee config/tp1.properties; Simulation.execute(config) usa exclusivamente la configuración recibida.

## Anexo B. Reproducción del informe y de los experimentos

Generar el PDF del informe (desde docs/):

```
pandoc informe.md -o informe.pdf --pdf-engine=typst
```

[[TODO: documentar el uso de tools/analysis/run-experiments.ps1, las variantes de configuración y de dónde salen las tablas de la sección 10.]]